Поширено: yehor. Це копія чату між Claude та користувачем yehor. Вміст може містити неперевірену або небезпечну інформацію, яка не відображає погляди компанії Anthropic. Поширений знімок може містити вкладення та дані, які тут не показано.

Поскаржитися

Файли приховуються, коли надано спільний доступ

Маю технічне завдання з ось такими умовами, поки надсилаю файли, щоб ти їх записав в контекст і проаналізував, які кроки для вирішення нам потрібні. Згодом при їх виконанні будеш детально описувати чого обрав саме ці кроки і детально пояснював рішення. З першого кроку код не пиши, спочатку виберемо параметри і узгодимо як це розв'язати.

Прочитав усі три файли, я бачу таке. Код `html2json` треба написати в `html2json.js`, а `index.html` лишається без змін. Головні обмеження: **не можна використовувати DOM-парсер** (`DOMParser`, `innerHTML`, `createElement` для розбору), код **не має падати** ні за яких умов, і треба показати, що ти продумав крайні випадки. Тому нам потрібен власний токенайзер і збирач дерева. Нижче параметри, які треба узгодити, і мої рекомендації.

1. Структура JSON (найважливіше рішення)

Є три варіанти:

- **A. Уніфіковані вузли.** Кожен вузол — об'єкт з полем `type`. Елемент має вигляд `{type:"element", tag, attributes:{}, children:[]}`, текст — `{type:"text", value}`, також є `comment` і `doctype`. Порядок змішаного вмісту зберігається (`<p>Привіт <b>світ</b>!</p>`), структура однакова на будь-якому рівні вкладеності, а парсити такий JSON легко.
- **B. Компактний вигляд** `{ "div": [...] }`. Він читається краще, але дублікати ключів і порядок змішаного вмісту ламаються.
- **C. Стиль xml2js** з `_` і `$`. Він менш прозорий, і його складніше пояснити.

Рекомендую A. Він єдиний нічого не втрачає, а на code review його найлегше обґрунтувати. Кореневий вузол — `{type:"document", children:[...]}`. Це потрібно, бо приклад 2 є фрагментом із кількома можливими кореневими вузлами, а приклад 1 — повним документом.

2. Крайні випадки, які парсер має витримати

- **Void-елементи** (`meta`, `link`, `br`, `img`, `input`, `hr` тощо) не мають закривного тега, а `<br/>` і `<div/>` треба обробляти окремо.
- **Атрибути:** подвійні, одинарні лапки та без лапок; булеві (`disabled`); дублікати (за специфікацією перемагає перший); `>` усередині значення; регістр імен.
- **Raw text елементи** (`script`, `style`, `textarea`, `title`). Їхній вміст не парситься як HTML, інакше `if (a < b)` зламає розбір.
- **Коментарі**, `<!DOCTYPE>`, `<![CDATA[`, незакриті коментарі.
- **Зламаний HTML:** закривний тег без відкривного, незакриті теги, перехресна вкладеність (`<b><i></b></i>`), одинокий `<` у тексті, обірваний тег наприкінці вводу (`<div cla`).
- **Нетипові входи:** порожній рядок, лише пробіли, не-рядок (`null`, `undefined`, число).
- **Масштаб:** дуже довгий текст, як у прикладі 2, і дуже велика глибина вкладеності. Рекурсивний збирач дерева тут може переповнити стек, тому збирати дерево слід **ітеративно, через явний стек**. `JSON.stringify` теж рекурсивний, тому для патологічної глибини (тисячі рівнів) потрібне обмеження глибини.
- **Продуктивність:** один прохід за лінійний час і жодних регулярок зі складним поверненням.

3. Рішення, які я хочу узгодити з тобою

1. **Пробіли.** Пропоную відкидати текстові вузли, що складаються лише з пробілів між елементами, і зберігати текст усередині `pre`, `textarea`, `script`, `style` без змін. Решту тексту можна лишати як є або обрізати по краях.
2. **HTML-ентіті** (`©`, `&`, `©`). Варіанти: залишати сирими, або декодувати невеликий набір найпоширеніших плюс числові. Я б декодував мінімальний набір, а невідомі лишав як є.
3. **Булеві атрибути:** значення `""` (як у специфікації) чи `true`.
4. **Імена тегів і атрибутів:** приводити до нижнього регістру чи зберігати оригінал. Я б нормалізував до нижнього.
5. **Неявні закриття** (`<li>`, `<p>` без закриття). Можна робити простий варіант: закрити найближчий відкритий збіг при зустрічі закривного тега та закрити все при кінці вводу. Повні правила HTML5 тут надмірні.
6. **Поведінка при помилці.** Функція ніколи не кидає виняток. Варіант: повернути найкраще можливе дерево разом з необов'язковим полем `warnings` (наприклад, «закривний тег без пари»). Це добре показує продуманість.

4. План робіт

1. Узгодити параметри вище.
2. Зафіксувати специфікацію формату JSON (приклад на невеликому вводі).
3. Написати токенайзер (текст, відкривний/закривний тег, коментар, doctype, raw text).
4. Написати ітеративний збирач дерева з обробкою помилок.

5. Створити `html_samples/`: приклади 1 і 2, порожній ввід, зламаний HTML, великий файл, глибока вкладеність, `script` з `<`, атрибути в усіх формах.
6. Протестувати в Node без браузера й у `index.html`, потім оновити `README.md` з поясненням рішень.
7. Підготувати `ai_help/`. Вимога каже про посилання на чат ChatGPT, а для інших ШІ — pdf/png/mp4 з історією діалогу, тож для цього чату знадобиться експорт чи скріншоти.

Які варіанти обираємо в пунктах 1–6? Якщо тебе влаштовують усі мої рекомендації, напиши «погоджую», і я перейду до кроку 2 — специфікації формату JSON.

1) я думаю залишати ці теги, бо вони все таки є в структурі, то нехай будуть, обрізати не треба
2) давай декодувати мінімальний набір
3) давай як в специфікації
4) давай нормалізуєм до нижнього
5) давай закрити все при кінці вводу
6) давай так і зробим, додамо на головну сторінку блок в якому ми надсилаємо файл для парсингу, а також поле для виведення отаких попереджень, котре буде їх показувати в результаті, також додати можливість збереження результуючого джейсона і перегляд його також
згодом думаю на безкоштовному хостингу це підняти, для наглядності

Усе зрозуміло, ваші відповіді прийняті. Нижче фіксую рішення разом з наслідками, потім специфікацію формату (кроку 2 плану), а наприкінці кілька питань, які ще треба закрити перед кодом.

Що змінюється через ваші рішення

Пробіли (п. 1). Не обрізаємо й не відкидаємо текстові вузли з самих пробілів. У прикладі 1 JSON буде повний вузлів на кшталт `{"type":"text","value":"\n  "}`. Це довше, зате це точне відображення вхідного HTML без втрат, і таке рішення легко захистити на code review: перетворення зворотне, нічого не вгадується. Якщо потім захочеться компактності, це можна додати окремою опцією.

Закриття тегів (п. 5). Реалізую так: коли зустрічається закривний тег, шукаємо його збіг у стеку відкритих елементів. Знайшли — закриваємо все до нього включно, а проміжні елементи закриваються неявно (так обробляється `<b><i></b>`). Не знайшли — тег ігноруємо й додаємо попередження. Усе, що лишилось відкритим у кінці вводу, закривається автоматично, і теж з попередженнями. Правил HTML5 для `<p>` та `<li>` не робимо.

Попередження (п. 6). Вони потрапляють у результат, тому функція ніколи не кидає виняток, а проблеми повертає як дані.

Специфікація формату JSON

Корінь:

```json
{
  "type": "document",
  "children": [ ... ],
  "warnings": [
    { "code": "UNMATCHED_CLOSING_TAG", "message": "Closing tag </div> has no matc
  ]
}
```

Чотири типи вузлів:

type	поля
`element`	`tag`, `attributes` (об'єкт «ім'я → рядок»), `children`
`text`	`value`
`comment`	`value`
`doctype`	`value` (наприклад, `"html"`)

Приклад для `<p class="a" hidden>Hi <br> there</p>`:

```json
{
  "type": "element", "tag": "p",
  "attributes": { "class": "a", "hidden": "" },
  "children": [
    { "type": "text", "value": "Hi " },
    { "type": "element", "tag": "br", "attributes": {}, "children": [] },
    { "type": "text", "value": " there" }
  ]
}
```

Чому саме так:

- `children` **завжди масив, навіть у void-елементів.** Споживачеві не треба перевіряти, чи поле існує.
- `warnings` **завжди присутнє**, порожнє, якщо проблем немає. Структура корня стабільна.

- `line` і `column` допомагають знайти проблему у великому файлі. Рахуються за зміщенням під час токенізації, що майже не коштує нічого.
- **Атрибути як об'єкт, а не масив.** Читати так зручніше. Платимо за це втратою дублікатів, але за специфікацією HTML перший виграє, тож це не втрата, а правильна поведінка (з попередженням `DUPLICATE_ATTRIBUTE`).
- **Об'єкт атрибутів створюємо без прототипа.** Інакше атрибут з іменем `__proto__` мовчки зламає результат. Це один із тих «валідний HTML, а код поводиться дивно» випадків, про які говорить README.
- **Захист від глибини.** Збирання дерева в мене ітеративне, але `JSON.stringify` рекурсивний і падає на дуже глибокій вкладеності. Пропоную ліміт, наприклад 500 рівнів: глибші елементи прикріплюються до останнього дозволеного предка з попередженням `MAX_DEPTH_EXCEEDED`.

Що ще треба узгодити

1. `<div/>` **з похилою рискою на не-void елементі.** За HTML5 слеш ігнорується і елемент лишається відкритим. Для практичного конвертера я б трактував це як закритий порожній елемент (так працюють SVG і JSX, і так очікує більшість людей), без попередження. Погоджуєтесь?
2. **Сутності.** Мінімальний набір: `& < > " '   ©`, числові `©` та `©`. Невідомі лишаємо як є. Декодуємо в тексті й у значеннях атрибутів, у `textarea` і `title` теж, а в `script` і `style` ні. Підходить?
3. `<![CDATA[...]]>` **у звичайному HTML.** За специфікацією це «хибний коментар». Пропоную зберігати його як вузол `comment`.
4. **Файл** `index.html`**.** README дозволяє його не змінювати і вимагає включити в архів. Не хочу ризикувати: пропоную **лишити оригінальний** `index.html` **недоторканим** і зробити окрему демо-сторінку `demo.html` для хостингу, з тим самим `html2json.js`. Тоді перевіряльники відкриють оригінал, а у вас буде гарна версія для показу. Для цього в `html2json.js` має лишитися сумісність з оригіналом (`convertHtml2JsonAndSet`, `showExample1`, `showExample2`), а експорт для Node додається під перевіркою `typeof module`.
5. **Демо-сторінка.** Пропоную: вибір файлу (або вставка тексту), кнопка конвертації, панель попереджень зі списком і позицією, перегляд результату, кнопка «Зберегти JSON» (завантаження `.json`). Для хостингу підійде GitHub Pages: сторінка чисто статична, і шаблон завдання вже на ньому. Підійдуть і Netlify чи Cloudflare Pages. Для дуже великих файлів `textarea` гальмує, тому для виводу, мабуть, краще окрема область, але це вже деталь кроку 6.

Якщо відповідь «так» на пункти 1–4 (і за демо-сторінку окремим файлом), я перейду до кроку 3: написання токенайзера, з поясненням кожного рішення. Якщо хочете щось змінити, скажіть.

1) так
2) так
3) давай
4) тоді так зробимо окрему сторінку, тільки ще стилі напиши під нього в окремому файлі, щоб це виглядало пристойно
5) по кнопкам ок, хостинг оберемо як буде готовий код
виходить все погодили, тоді я обираю опус версію для його написання і чекаю на код проєкту

Усе узгоджено, пишу проєкт: парсер, окрему демо-сторінку зі стилями, набір HTML-зразків і автотести. Потім перевірю все в Node і в браузері.

Створено файлів: 10, виконано команд: 4, оновлено завдання

Зразки готові. Дивлюсь, що парсер видає на найкаверзніших із них:

Виконано команду, відредаговано файл

Тепер автотести. Беру вбудований `node:test`, щоб проєкт не мав жодних залежностей.

Створено файл, виконано команду

In Node 22 a directory isn't accepted as an argument, so I'll pass the file directly and fix the command in the docs accordingly:

Виконано команду

All 46 tests pass. Let me check how much headroom the performance tests have:

Виконано команд: 5, оновлено завдання

Now the demo page: markup, styles in a separate file, and the logic in a separate file as well.

Створено файлів: 3

Now I'll check both pages in a real browser: screenshots, console errors, loading a file, saving the JSON, clicking a warning, and the original `index.html`.

Виконано команду

Everything works, with no errors in the console. Now let me look at how it looks:

Відредаговано файлів: 2, прочитано файлів: 5, виконано команду

Both themes and mobile look good now. Next: README with the reasoning, `ai_help/`, and a final full run.

Оновлено завдання, створено файл

Fixing a typo I noticed ("fragmentsn"), then `package.json` for `npm test` and the `ai_help` folder:

Створено файлів: 2, відредаговано файл

Final check: the full test suite, that `index.html` is byte-for-byte identical to the original, and packaging into an archive.

Виконано команду, оновлено завдання, спільні файли

The project is ready, in the archive `html2json.zip`. The parser, demo page, samples and tests have been checked: all 46 tests pass, and the original `index.html` is byte-for-byte unchanged.

How the parser works and why

Two passes instead of one big recursive function. The first pass is the tokenizer (`class Tokenizer`). It walks the string from left to right once and produces a flat list of tokens: text, opening tag with attributes, closing tag, comment, doctype. The second pass is the tree builder (`buildTree`), which assembles these tokens into a tree. I split them because each part does one job: the tokenizer knows nothing about nesting, and the builder knows nothing about quotes and `<`. That's easier to test and easier to explain on a code review. Browsers are built the same way, by the HTML specification.

The builder uses an explicit stack, not recursion. A recursive parser crashes with "Maximum call stack size exceeded" on a few thousand nested `<div>`s, and the task says directly that the code must not crash. With an explicit stack, 100,000 levels of nesting parse in half a second.

A depth limit of 500. This is a subtle point: even if our parser survives deep nesting, `JSON.stringify` is recursive and will crash on the result itself. So past 500 levels, the content of deeper elements is placed next to them rather than inside. Nothing is lost, and there's a warning. Real pages are rarely deeper than 30–40.

Linear time even on malicious input. I covered the places where a naive implementation slows down quadratically. An unmatched closing tag is checked in O(1) through a counter of open tags, instead of walking the whole stack. Line and column are computed only when the first warning appears, using binary search. There are no regular expressions with catastrophic backtracking. A 10 MB page converts in about 2 seconds.

Edge cases that are easy to miss. These are worth bringing up on a review. An attribute named `__proto__` on an ordinary object silently disappears, so the attributes object is created without a prototype. `&constructor;` with a plain-object lookup table would "decode" into a function, so the table is a `Map`. `&#0;` and surrogate code points make `String.fromCodePoint` throw, so they become `&#65533;` as in browsers. Lowercasing with `toLowerCase` changes `İ` into two characters, so only ASCII is lowercased, as the specification requires.

How I verified it

Besides the ordinary tests, there are three stronger checks.

1. **Round trip:** 2,000 random trees are converted to HTML and parsed back, and the result must match exactly.
2. **Fuzzing:** 5,000 random pieces of garbage made of `<`, `</`, `<!--`, quotes, `&#` must not crash the function.
3. **Comparison with Chromium's real parser.** For the task's Example 1 and the 6,000-row table, our tree is identical to the browser's. The other differences appear only where we deliberately chose differently (`<li>` without a closing tag, `<div/>`, rare entities like `—`). They are listed in the README as known limitations, with the reasons.

What's in the archive

What's in the archive

- `html2json.js` contains the implementation. The functions for the original `index.html` are kept, so its buttons now show real results.
- `demo.html` + `demo.css` + `demo.js` is the extended page. You can open a file or drag one in, see warnings (clicking one highlights the place in the input), browse the result as a tree or JSON, and copy or save the `.json`. It supports a dark theme and mobile. The DOM is used only to display results; parsing goes through our function, and user text is always inserted with `textContent` (no XSS).
- `html_samples/` holds 16 files, from an empty one to 0.8 MB and 2,000 levels of nesting.
- `tests/`: run with `npm test`, no dependencies.
- `README.md` is in English, since Jito will read it. It covers the format, every decision with its reasoning, the warning codes, limitations and testing.

What you need to do yourself

In `ai_help/claude_chat.txt`, insert the public link to this chat in place of `PASTE_SHARE_LINK_HERE`. Also add `claude_chat.pdf`, an export or printout of the conversation, because the task asks for `.pdf/.png/.mp4` for AI tools other than ChatGPT. Check that the link opens in incognito mode, as the README warns.

The next step is hosting. I'd suggest GitHub Pages: the task's template is already hosted there, and the site is purely static. Let me know when you're ready, and we'll set it up and add the link to the README.


```
{
  "type": "document",
  "children": [
    {
      "type": "element",
      "tag": "div",
      "attributes": {},
      "children": [
```


Показати більше

First, the explanation, then I'll redo the viewer.

Where the `\n` nodes come from. They aren't added by the parser. They are the line breaks and indentation from the source HTML itself. Example 2 is written like this:

```
<div>↵
<p>Hello world!</p>↵
␣␣<button>…
```

Between `<div>` and `<p>` there is a line break: that's the first `"\n"`. Between `</p>` and `<button>` there is a line break plus two spaces: that's `"\n  "`. The final `"\n"` after `</div>` is the line break at the end of the file. For the parser this is ordinary text, the same as `Hello world!`; it just happens to consist only of whitespace. In the first step we agreed to keep such nodes and not trim them (decision 1). That way the JSON reflects the input exactly and nothing gets "guessed". The browser does the same: `div.childNodes` in Example 2 also has 7 nodes, including these text ones. Keeping them is a sound choice that's easy to defend on a review. If you'd still prefer a cleaner result, I can add an option such as `html2json(html, { ignoreWhitespace: true })` that drops nodes made only of whitespace (except inside `pre`, `textarea`, `script`, `style`). Tell me if you want it.

The viewer. That's fair: the tree was a different representation, not the JSON itself. I'll remove it and show exactly the text that ends up in the saved file, with syntax highlighting.

Відредаговано файл

Now the logic: I'm removing all the tree code and replacing it with a JSON viewer that highlights keys, strings, numbers and `null/true/false`.

Виконано команд: 2

The regex is correct: escaped quotes inside strings and a `:` inside a key don't break it. Now the styles: I'm removing the tree and tab styles and adding colors for JSON.

Виконано команду

Checking that the CSS is intact and the page works:

Виконано команду

The page works, but grep didn't find `.btn--ghost` and `.shortcut`. I suspect my cut matched the earlier `.toolbar__actions .checkbox {` and removed too much. Checking:

Виконано команду

Yes, the cut removed all the button styles. Restoring them (without the checkbox, which is no longer used):

Edited a file

There are also two `.json-view` rules in a row (left over from `.json-view, .tree`). Merging them into one:

Виконано команду, відредаговано файл

Now I'll update the README (it mentioned the tree view), re-run the browser check and the tests, and look at the screenshot:

Прочитано файлів: 2, виконано команду

It all looks right. Rebuilding the archive:

Виконано команду, спільні файли

The demo page's viewer now shows the JSON itself, not the tree. The "Tree/JSON" tabs and the "Show whitespace" checkbox are gone. I checked that the text in the viewer matches the file "Save .json" saves character for character. Keys, strings, numbers and `true/false/null` are highlighted in different colors. For very large results (over 2 MB of JSON), the highlighting turns off automatically and the text is shown plain, so the page doesn't freeze; on the 0.8 MB table that's about 12 MB of JSON.

I also removed the "tree view" mentions from the README and added a note that the browser keeps these whitespace text nodes too. All 46 tests still pass.

On the `\n`: if you decide you want a more compact result after all, say so, and I'll add the `ignoreWhitespace` option.

Файли приховуються, коли надано спільний доступ

а чи релевантно це додавати в парсер, саме оці пробіли, бо вони просто додаються після кожного елементу ось для прикладу
ось приклад перший і в результаті ці переходи рядка навіть після тегу doctype і html і взагалі після всіх

You're right: in this form they're mostly noise. In Example 1 the JSON has 45 text nodes, and only 8 of them carry real text ("Sample HTML", "Home", "Welcome to My Website"...). The other 37 are line breaks with indentation, i.e. formatting of the source file, not page content. They make the JSON longer and harder to read, and anyone using this JSON would have to filter them out by hand. When we agreed to keep them, I emphasized that this is lossless, but in practice someone converting HTML to JSON wants the structure, not the indentation from the editor.

They can't simply be dropped across the board, though, because in some places whitespace matters:

- In `<b>Hello</b> <i>world</i>` the space between the words is content. Without it the text becomes "Helloworld".
- Inside `<pre>`, `<textarea>`, `<script>`, `<style>` every space and line break is significant.

So I suggest the rule JSX in React uses, which is time-tested: **a whitespace-only text node is dropped if it contains a line break, and kept if it's spaces on the same line**. Indentation always comes after a line break, so it disappears, while the space between inline elements on the same line stays. Inside `pre`, `textarea`, `script`, `style` nothing is touched. Text with real words isn't trimmed: `" Hello "` stays as it is. Example 2 would then be just `div → p, button, textarea`, with no `\n`.

There's one trade-off, and I'll describe it honestly in the README: if two inline elements are on different lines (`<b>a</b>↵<b>b</b>`), the browser renders a space between them, but the JSON won't have one. JSX has exactly the same behavior, and it's an accepted compromise.

For flexibility, I'd add an option `html2json(html, { preserveWhitespace: true })` that brings back the current full output, plus a "Keep whitespace" checkbox on the demo page. The default would be the clean version. The function signature stays compatible with the task, since the second argument is optional.

Shall we go with this? If yes, I'll make the changes, update the tests (including the round-trip test) and the README.

> Так, давай з цим варіантом: правило про пробіли, нічого не чіпаємо в pre/textarea/script/style, і опція preserveWhitespace, і додай відповідні тести.

Agreed. Plan: the decision about dropping is made in the tree builder, but only after a text node is fully assembled. A text node can be stitched together from pieces (`a</x>` + `\n`), and the decision must be made for the whole node, not for the pieces. Starting with the parser:

Відредаговано файл, прочитав файл

Now a helper that checks "is this formatting whitespace":

Відредаговано файл, прочитав файл

I'm adding to the builder a counter of open "sensitive" elements and a list of candidates, filtered once at the end:

Відредаговано файл, прочитав файл

If reading the options throws (a getter on the object), it's better not to lose the whole result to `INTERNAL_ERROR` but to fall back to defaults with a warning. Adding that:

Відредаговано файлів: 2, виконано команду

Small improvements to the tests: add `pre` to the random trees so the round-trip also exercises the "sensitive" branch, run the fuzz test in both modes, and make the Example 2 test non-circular:

Виконано команду

All 58 tests pass. Now the demo page: a "Keep whitespace" checkbox next to the Convert button.

Відредаговано файлів: 2, виконано пошук у файлах, виконано команду

Checking in the browser: the checkbox toggles the mode, the viewer equals the file, and the mobile layout doesn't break:

Прочитано файлів: 2, виконано команду

On mobile the Convert button is clipped on the right (there's a "Ctrl Enter" hint that's useless there anyway). I'll hide the hint on narrow screens and allow the footer to wrap:

Відредаговано файл, виконано команду, прочитав файл

It now fits, but the hint is still showing: the base `.shortcut` rule comes later in the file and overrides the media query. Moving the hide rule after it:

Виконано команду

Now the README: the whitespace section, the function's signature, the new warning, the limitation, and the testing figures.

Відредаговано файл, прочитано файлів: 2, виконано команду

Re-running the browser comparison (to make sure the README claim still holds), the full test suite, and the check of the original `index.html`:

Виконано команду, спільні файли

Formatting whitespace is now dropped by default. Example 2 comes out clean (`div` → `p` , `button` , `textarea` , no `\n`), and in Example 1 there are 10 text nodes instead of 45, all with real text. All 58 tests pass, and the updated archive is `html2json.zip` .

Where the drop decision is made. You can't decide immediately when a text piece shows up. In `<p>a</x>↵  </p>` , the parser first sees `a` , then ignores `</x>` , then sees `↵  ` , and stitches all of it into one node `"a\n  "` . If the decision were made per piece, `↵  ` would be thrown out and the text would lose part of itself. So during building, the builder only records "candidates", and after the tree is assembled it checks each one by its final value. Removal goes through a `Set` , and each `children` array is compacted once, so the cost stays linear.

How pre/textarea/script/style are protected. The builder keeps a counter of open "sensitive" elements: it goes up when such a tag opens and down when it closes. While the counter is above zero, text doesn't become a candidate. That's why `<pre>↵<b>x</b>↵</pre>` keeps both line breaks around `<b>` , even though they're directly inside `<b>` 's parent and not in `pre` itself. A `pre` left unclosed until the end of the file is protected to the end. I also added `title` and the rarer raw elements to this list, since their content is a single text node too.

The `preserveWhitespace` option is the second, optional argument, so the `html2json(htmlText)` signature from the task still works unchanged. Invalid values (a string instead of an object, `"true"` instead of `true` , a getter that throws) don't crash the function. It uses the defaults and adds an `INVALID_OPTIONS` warning, in keeping with the "never throws" principle.

The tests (12 new):

- Indentation is removed, a space on one line is kept, text with words isn't trimmed, and ` ` counts as content.
- `pre` / `textarea` / `script` / `style` / `title` are protected, including nested and unclosed cases.
- The case with merged text nodes, both modes, invalid options, and both examples from the task.

The round-trip test now runs twice. With `preserveWhitespace: true` the result must match the source tree exactly; in the default mode it must match the source tree minus the formatting whitespace. I added `pre` to the random trees so the protected branch gets exercised too. The fuzz test also runs in both modes.

On the demo page there's a "Keep whitespace" checkbox next to the Convert button. Toggling it re-converts immediately. While checking on mobile I found the Convert button was being clipped and fixed that.

In the README I rewrote the whitespace section with the reasoning and the examples' figures, added an API section and the `INVALID_OPTIONS` code, and described the trade-off as a known limitation: two inline elements on different lines lose the space between them, the same as in JSX. The Chromium comparison still shows full agreement for Example 1 and the large table (in `preserveWhitespace: true` mode).

тепер попробуєм зламати цей код за допомогою прикладів з підковиркою, якісь занадто довгі чи короткі файли і подібне, також скинь їх мені, щоб я їх руками протестив в браузері, потім весь їх вивід я додатково скину тобі щоб ти проконтролював відповідно алгоритму чи вірно працює програма

The plan: first I'll try to break it myself (run all the samples in Node and in a browser via Playwright), fix whatever turns up, and then send you the set for manual testing.

Виконано команд: 2

Run everything through the parser in Node: whether it crashes, timings, JSON size, and the shape of the result:

Виконано команд: 3, відредаговано файл, виконано пошук у файлах

Checking how accurate the estimate is compared with the real difference between pretty and compact JSON, and what each stress file now gets:

Виконано команд: 2, прочитав файл, виконано пошук у файлах

A note about compact mode can be long, so I'll let the label wrap neatly on the right, then repeat the browser stress run:

Виконано команд: 7, відредаговано файлів: 2

63 tests, all pass. Now I'll document the findings in the README and add a short description of each stress file to the folder:

Відредаговано файл, прочитано файлів: 2, створено файл

Final run and packaging. I'll make two archives: the whole project, and the stress files separately so they're convenient for manual testing:

Виконано команду, спільні файли

I wrote 37 files designed to break the converter. The parser itself survived all of them: no file made `html2json` crash. But the code that displays the result broke in three places, and all three are fixed.

What broke and why:

1. **The `index.html` page threw an exception on deep nesting.** This is the most important finding, because it's the page the reviewers will open. On 100,000 nested `<div>` s the parser takes 0.2 s, but the pretty-printing `JSON.stringify(result, null, 2)` dies with `RangeError: Invalid string length`. Every line of the pretty output repeats indentation proportional to its depth: at a depth of 500 that's ~2,000 spaces per line, times 100,000 elements, roughly a gigabyte of spaces. Even the harmless 5 KB file with depth 501 turned into 3.5 MB of JSON.
    - **The fix:** before pretty-printing, the code estimates the size of the indentation by walking the tree (each line at level L costs `1 + 2·L` characters, accurate to within a few percent). If it's over 32 MB, it outputs compact JSON, which has no indentation and grows linearly. Simply catching the error and retrying wasn't enough: the failed attempt itself took 4 s and a gigabyte of memory.
2. **The browser tab crashed on the 10 MB table.** That's 115 MB of pretty JSON, and inserting that much text into a field kills the tab. Now the page shows at most 5 MB with an explicit note, while "Save .json" in the demo writes the full result. I checked: a valid 38 MB JSON with all 116,279 rows.
3. **UTF-16 and Windows-1251 files turned into garbage in the demo.** The parser isn't at fault here: the demo read every file as UTF-8, so the string was already broken when it reached the parser. The demo now picks the encoding the way browsers do: BOM, then a UTF-8 check, then `<meta charset>`. If none of that works, it says honestly that some bytes were replaced.

I also checked the correctness of the results: I compared the tricky files with Chromium's built-in parser. The only differences are the ones we chose deliberately, plus one I noticed: the browser throws `\u0000` out of text, while we keep it. Some results look strange but are correct per the specification. `<привіт>` becomes text, because a tag must start with a Latin letter. `&#x1F60` without `;` at the end of the file becomes the letter "ὠ". When you test these files, don't be surprised: that's exactly how a browser behaves.

The tests: 63 (5 new), all pass. One of them runs the actual `convertHtml2JsonAndSet()` from `index.html` against a stand-in `document` and checks that it doesn't throw on huge or deep input. The stress files are in the project, in `html_samples/stress/`, with a README describing what each one attacks and what it found.

What to send me after your manual run:

- **Small files:** the full JSON from the demo ("Copy" or "Save .json"), along with the warnings.
- **Large ones** (30–45): the stats line (number of elements, time), the note next to "Result JSON", and the list of warnings.

Load the files through "Open file" so encoding detection kicks in. You can also run a few through the original `index.html` (paste the text into the field): the big ones should now show the beginning of the JSON with a note rather than freezing.

Файли приховуються, коли надано спільний доступ

перші файли для перевірки результатів надсилаю тобі

I'll check your results against two references: first a mechanical byte-for-byte comparison with what the parser produces from the original files, then the meaning of each result against the algorithm's rules.

Виконано команду

All 15 files match the reference byte for byte. I also went through the meaning of each one against the algorithm's rules, and they're correct. Here's what's worth noticing in each:

Single characters (01–07)

- **01–03:** `<`, `>`, `&` stay text, because they don't start a tag or a valid reference.
- **04, 05:** a file with only a BOM or only line breaks gives an empty `children`. The BOM is stripped by the preprocessing, and the line breaks are formatting whitespace.
- **06:** a lone `</` at the end of the file stays as text.
- **07:** the cases split up correctly:
  - `<><>` is text.
  - `</>` is ignored with a warning, and its column 5 matches the position in the file.
  - `</ >`, `<!>`, `<?>` became comments.
  - `<!---->` is an empty comment.
  - This file matched Chromium exactly in my earlier comparison.

Tag syntax (10)

- `<div<span>` became a tag named `div<span`, and `<a =x>` an attribute named `=x`. Both are exactly what the HTML specification prescribes.
- `<привіт>` and `<__proto__>` stayed as text, because a tag must start with a Latin letter. Their closing tags became comments, again just like the browser.
- `<constructor>`, `hasOwnProperty`, `toString` became ordinary tags and attributes and didn't break anything. That's what the prototype-free object and the `Map` are for.

Raw text (11)

- The fake `"</scr" + "ipt>"` in a string didn't end the script.
- `</ScRiPt  >` with mixed case and spaces did end it.
- `<!--` in the second script didn't swallow the rest of the file.
- `</ title>` with a space isn't a closing tag, so it stayed text inside `title`.
- The broken `<scr<script>` was left unclosed until the end of the file, and everything after it nested inside it, with two honest warnings.

Comments (12)

- `<!-- nested <!-- comment --> outer -->` ended at the first `-->`, and ` outer -->` became text. This is correct: HTML comments don't nest.
- CDATA and `<?php ...?>` are cut at the first `>`, which is spec behavior.
- The unclosed comment at the end swallowed the `<p>` that follows it, with an `UNTERMINATED_COMMENT` warning.

Entities (13)

- `&` decoded once, to `&`.

- `A` and `A` became `A`.
- Everything beyond Unicode or half a surrogate pair became `&#xFFFD;`.
- `&#x10FFFF;` is a valid code point, so it's kept (it shows as an empty square).
- `&`, `©` and ` ` without `;` stayed as written. That's our documented minimal set; a browser would decode them.

Structure (14): every closing tag closed its nearest open match. Every warning points to the right line and column (I checked several against the file). `<td>`, `<li>` and `div` in `p` are nested rather than closed per HTML5 rules, which is the known limitation we agreed on.

Whitespace (15)

- `<p>  </p>` kept its three spaces, because there's no line break.
- `<p>‍</p>` became empty.
- The space and tab between `<b>`s on one line were kept, while the line break between `<b>c</b>` and `<b>d</b>` was dropped. That's exactly the trade-off described in the README.
- Inside `pre`, the nested `pre`, `textarea` and `script`, everything is intact.
- ` `, the em space and `\f\v` stayed as content.
- Of the three `</pre>`, exactly one is unmatched, with a warning on line 14.

Line endings (16): CRLF, CR and LF gave the same result. The `</span>` warning is on line 6, which means a lone CR is counted as a line break too.

Unicode (17): RTL text, the zero-width characters and emoji sequences are intact. The BOM in the middle of the text is preserved (only a leading BOM is removed). `\u0000` stays: Chromium throws it out, we keep it, and that's documented.

No bugs, no unexpected behavior. Send the next batch when you're ready:

- **18–27:** the encodings and the files that end mid-token. Load these through "Open file", since the label next to the file name shows which encoding was detected.
- **30–45:** the large ones. For these, the stats line, the note next to "Result JSON" and the warnings are enough.

Файли приховуються, коли надано спільний доступ

наступні

Again I'll compare every file mechanically, this time decoding the HTML the same way the demo does (BOM, UTF-8, `<meta charset>`), since the encoding files are in this batch. For each file I check both modes in case "Keep whitespace" was on:

Виконано команду

My regex for the "1" suffix (for the file you sent twice) also stripped the "1" from "501". That's a bug in my checking script, not in the project. Fixing it and finishing the remaining ones:

Виконано команд: 2, виконано пошук у файлах, відредаговано файл

Tokenizer and `readOptions` just get their own stream (they only call `.add`). Inside the builder there are two streams, one for the token pass and one for the final unclosed elements:

Виконано команд: 3, відредаговано файл

As expected, of the 32 files you checked, only `23` changed. Everything else, including `32` with its 100,000 warnings, gives exactly the same result as before. Packaging:

Виконано команду, спільні файли

All 17 files in this batch match the reference byte for byte. Your repeated `18b` and `32` are identical to each other too, so the result is stable across runs. Checking the meaning did turn up one flaw, which is fixed and is in the `html2json.zip` archive.

What's correct and why:

- **18, 18b (UTF-16 LE and BE):** "Привіт" came through intact. The demo recognized the encoding by its BOM.
- **19b (Windows-1251 with `<meta charset>`):** all the Ukrainian letters are there, including Ґ, Є, Ї.
- **19 (Windows-1251 without a declaration):** turned into `<meta charset>`, the encoding can't be determined, and the browser would do the same. The demo honestly warned about the replaced bytes.
- **20 (binary garbage):** no crash, and the bytes that were "accidentally" HTML-like became elements and comments.
- **21, 22 (JSON and CSS):** the `<div>` inside a JSON string and the `<b>` inside a CSS `content` became tags. This is correct: the parser doesn't know it's been given something that isn't HTML, and parses it as HTML.
- **24, 25 (a file that ends inside a tag):** the cut-off tag stays as text, per our rule "don't silently lose input data". The warning points to the exact column where the tag starts.
- **26:** the unclosed `textarea` took everything up to the end of the file.
- **27:** `&#x1F60` at the end of the file became "ὠ", as discussed before.
- **30, 31, 32 (depth):**
  - 499 levels give no warnings.
  - 501 levels give `MAX_DEPTH_EXCEEDED`, and the content of the deepest elements moved up a level instead of being lost.
  - 100,000 levels: `"pretty": false`, compact JSON of 6.2 MB. Your "Save .json" saved the whole result, not the truncated 5 MB display.

The flaw I found in `23-many-warnings`. The README promises that the warning list keeps "the first 1,000". The file has 3,000 extra `</x>` first and only then 500 duplicate attributes. So the first 1,000 by position should all be `</x>`. But the list contained 500 `</x>` and 500 duplicates from the end of the file.

The cause is that warnings are created in different passes. The tokenizer runs first, so it took half the slots before the builder even started. Nothing was lost in the tree, and the "3000 more" counter was accurate, but the list itself was misleading.

The fix uses one property: each source produces warnings in increasing position order. Each source keeps its own first 1,000, then they're merged, sorted, and the overall first 1,000 are kept. The result is exactly the first 1,000 by position, and memory is still bounded. There's also a third source, the final list of unclosed elements. It goes separately because it reports earlier positions after later ones.

A new test covers both directions, with the problems at the start of the file and at the end. That makes 64 tests, all passing. I re-checked all 32 of your files against the new code, and only `23`

changed.

Next come files 33 and 40–45, the largest ones. For these, the stats line, the note next to "Result JSON" and the conversion time are enough. If you have time, also paste `45-big-table-10mb.html` into the original `index.html`: it should show the beginning of the JSON with a note that the output was truncated.

почекай поки я не скину всі файли, щоб вже в результаті того фіксити

Got it. From here on, when you send files I'll only check them and record what I find, without changing the code. Once you've sent everything, I'll give one list of all findings and we'll agree what to fix before I touch anything.

The fix for `23-many-warnings` is already in the last archive, though; I made it before your message arrived. Tell me if you'd rather hold it until the overall review, and I'll roll it back and add it to the list. Otherwise it stays, and I'll mark it as already fixed in the final list.

Send the remaining files (33, 40–45) when you're ready.

Файли приховуються, коли надано спільний доступ

ось остання пачка, вже після неї підсумуй мені результати українською бажано, для того щоб я все вловив і будем вже фіксити

This is the last batch, so I'll check it mechanically just like the others, without changing the code, and then put together the overall summary:

Виконано команду

All 37 files have been checked, and every result matches the reference byte for byte. Here's the overall summary.

The big files from the last batch

- `45-big-table-10mb`: your three saves are identical to each other and to the reference (38.8 MB of compact JSON, all 116,279 rows). The result is stable and the large file didn't break the page.
- `33` **(50,000 elements side by side):** all elements are in place, and the JSON stays pretty-printed because the tree is shallow.
- `32` **(100,000 nested elements):** compact JSON, and the content past depth 500 isn't lost.
- `40`, `41`, `44` **(5 MB on one line, a 2 MB attribute, an unclosed quote):** no slowdowns. The unclosed quote kept the whole tail as text, with a warning.
- `42`: all 10,000 attributes are in order, from `a0` to `a9999`.
- `43`: a million `<` characters became one text node, with no warnings.

What the testing found overall

Already fixed (when I ran the files myself before your manual test):

1. The original `index.html` threw an exception on deeply nested HTML because of pretty-printing. Now the code checks in advance whether pretty JSON is affordable, and otherwise outputs compact JSON.
2. The browser tab crashed on huge output. Now at most 5 MB is shown, and "Save .json" saves everything.
3. The demo read UTF-16 and Windows-1251 files as garbage. Now it detects the encoding the way a browser does.

Found while checking your results:

4. The warning cap kept "the first 1000 added" rather than the first 1000 by position in the file. This is already fixed (you asked to wait; tell me if you want it rolled back).

In the parser itself, none of the 37 files caused a crash or a mistake against our rules.

What's left to decide

These are differences from how a browser behaves. They're decisions we made consciously, but now is the moment to reconsider them.

#	Situation	How it works now	How a browser does it	My opinion
A	`<pre>↵text` or `<textarea>↵text`	the leading line break is kept	it's dropped	**Fix it.** The line break after `<pre>` is source formatting, and browsers deliberately ignore it. It's a few lines of code.

B	Entities `—`, `…`, `&`, `&copy` without `;`	stay as written	decoded	**Extend it.** Add ~30 of the most common ones (dashes, quotes, ©®™, €, arrows) plus the old forms without `;`. The full table of 2,231 entities would bloat the code.
C	`<li>a<li>b`, `<p>a<p>b`, `<td>a<td>b` without closing tags	the next one nests inside the previous one	the previous one is closed automatically	**Optional, worth doing.** This is common in real HTML, and right now the tree comes out "deeper" than it really is. I'd implement a short list of rules for `li`, `p`, `td/th`, `tr`, `option`, `dt/dd`, not the whole HTML5 algorithm.
D	A tag cut off at the end of the file (`<div cla`)	kept as text with a warning	silently dropped	**Leave it.** Our approach is more honest: nothing disappears, and there's a warning.
E	The `\u0000` character in text	kept	dropped	**Leave it.** It doesn't break JSON, and nothing is lost.
F	A line break between inline elements on different lines	dropped	rendered as a space	**Leave it.** This is the compromise from the JSX rule we chose.
G	Tags with non-Latin names (`<привіт>`)	text	text	Already behaves like the browser; nothing to do.

One more thing that isn't about correctness: the demo converts on the main thread, so on the 10 MB file the page "freezes" for about 4 seconds with no visual feedback. I can add a "Converting…" indicator. A Web Worker won't work here, because the browser blocks workers when the page is opened as a file from disk.

Tell me which items to fix (my recommendation is A, B, C and the indicator), and I'll make them in one go, with tests for each.

підсумуй мені результативність

Результати всіх 37 стрес-файлів + оригінальних прикладів — зведено.

Тестовано: 37 "ламаючих" файлів (крихітні/порожні входи, трюки з тегами/коментарями/

сутностями/структурою, різні кодування — UTF-16, Windows-1251, велике вкладення (100к), велика ширина (50к), файли по 5-10 МБ) + оригінальні 16 прикладів, у двох режимах пробілів. Усе перевірено вручну в браузері тобою + механічно порівняно мною з еталонним виводом.

Результат: сам парсер ніде не впав і не видав невірний результат.

Знайдені й виправлені баги (усі — у коді навколо парсера, не в самому алгоритмі):

1. **Крах на глибокій вкладеності** — `JSON.stringify(..., null, 2)` на 100 000 вкладених елементів вимагав ~1 ГБ пам'яті на відступи → `RangeError`. Знайдено до відправки тобі, виправлено через `stringifyResult()` (компактний JSON, коли відступи занадто дорогі).
2. **Крах вкладки на величезному тексті** — 10 МБ таблиця → 115 МБ JSON, браузер падав. Виправлено через `textForDisplay()` (показує перші 5 МБ, Save/Copy — повний файл).
3. **Кодування у демо** — UTF-16 і Windows-1251 читались як сміття. Виправлено через визначення кодування як у браузері (BOM → UTF-8 → `<meta charset>`).
4. **Ліміт попереджень не за позицією** — знайдено саме твоєю ручною перевіркою `23-many-warnings.html`: кешувались перші 1000 за порядком виконання проходів (токенізатор-білдер), а не за позицією в файлі. Вже виправлено через multi-stream колектор (ти просив зачекати з фіксами — цей встиг вийти до того, можу відкотити, якщо хочеш).

Усі інші файли (включно з останньою партією: 32, 33, 40-45) — точний збіг з еталоном, нових проблем нема.

Що лишається нефіксованим — розбіжності з поведінкою браузера, свідомі рішення, варто переглянути:

#	Ситуація	Зараз	Браузер	Моя думка
A	Перехід рядка одразу після `<pre>` / `<textarea>`	лишається	відкидається	**Виправити** — це форматування джерела, не контент
B	`—`, `…`, `&copy` без `;` тощо	не декодується	декодується	**Розширити** — додати ~30 найпоширеніших
C	`<li>a<li>b` без закриття	вкладається	авто-закриття	**Варто зробити** — короткий список правил для `li/p/td/tr/option/dd`
D	Обрізаний тег в кінці файлу	текст + warning	тихо зникає	Залишити — наш варіант чесніший
E	`\u0000` в тексті	лишається	відкидається	Залишити — нічого не ламає
F	Перехід рядка між inline-елементами	відкидається	стає пробілом	Залишити — наслідок обраного правила JSX
G	Теги з не-латиницею	текст	текст	Вже як у браузері

Плюс: конвертація на 10 МБ файлі блокує сторінку на ~4 сек без індикатора — можу додати "Converting…".

Моя рекомендація: виправити A, B, C + додати індикатор. Скажи, що фіксити — зроблю все разом з тестами.

фіксим те що ти рекомендуєш і будемо обирати хостинг, але поки підправляй, що потрібно і виходимо на фінал розробки

Виправляю A (перехід рядка після `<pre>` / `<textarea>`), B (розширені сутності), C (авто-закриття `li/p/td/…`) і додаю індикатор «Converting…», з тестами.

Оновлено завдання, виконано команд: 3, прочитано файлів: 3

Тепер вношу зміни в парсер.

Відредаговано файл, виконано команд: 2, створено файл

Тепер авто-закриття в побудовнику дерева.

Виконано команду

The safety net caught a slip (`input` undefined in readStartTag) — fixing.

Виконано команду

Очікувані падіння (старі тести фіксували стару поведінку). Дивлюсь кожне.

Виконано команд: 2

Старі тести оновлено, всі 64 проходять. Додаю нові тести для A, B, C.

Виконано команду

76/76. Тепер звіряю з реальним парсером Chromium.

Виконано команд: 5, створено файл

Тепер README. Знаходжу розділи, які треба оновити.

Виконано команд: 2, прочитав файл

Перевіряю твердження README про ідентичність з Chromium для Example 1 і таблиці на 6000 рядків.

Виконано команд: 2

Різниця в 04 — це `<textarea/>`, наше давнє свідоме рішення. Тепер індикатор у демо.

Виконано команд: 10, оновлено завдання, прочитано файлів: 3

Я виправив усе, що рекомендував. Усі 76 тестів проходять (раніше було 64). `index.html` лишився байт-у-байт оригінальним. Фінальна версія в архіві `html2json.zip`.

Що змінилось:

- **A. Перехід рядка після `<pre>`, `<listing>`, `<textarea>`.** Тепер він відкидається, як у браузері (`<pre>↵code` дає `"code"`). Галочка «Keep whitespace» (`preserveWhitespace: true`) зберігає його, бо цей режим тримає кожен символ джерела.
- **B. Сутності.** Тепер декодується близько 80 найпоширеніших: тире, лапки, `…`, `™`, `€`, стрілки, дроби тощо. Додав і правила браузера:
  - старі назви працюють без `;` (`&copy 2024` → `© 2024`);
  - в атрибутах `?a=1&copy=2` не ламається;
  - `—` → `—` (так звані Windows-1252 номери).

  Невідомі назви лишаються як є.
- **C. Авто-закриття.** Тепер `<li>a<li>b`, `<p>a<div>`, `<td>1<td>2`, `<tr>`, `<dt>/<dd>`, `<option>` закриваються як у браузері, з урахуванням меж: вкладений `<ul>` або таблиця в комірці — окрема зона. Попередження за цим правилом більше не з'являються, бо таке закриття — валідний HTML. Попередження лишається, лише коли дорогою довелось закрити щось інше (`<li><b>x<li>`). На крайніх випадках (50 000 `<li>` під 20 000 відкритих `<div>`) час залишається лінійним.
- **Індикатор у демо.** На файлах понад 200 КБ кнопка спершу показує «Converting…» зі спінером і лише потім починає роботу.

Що ще знайшов під час перевірки. На 10 МБ таблиці поле результату виглядало порожнім: компактний JSON — це один рядок на 5 МБ, і Chromium його просто не малює. Тепер компактний JSON переноситься на нові рядки. Ти цього, мабуть, не помітив, бо файл через Save був повний.

Звірка з Chromium. Усі нові правила дають той самий результат, що й браузер, на 25 тестових прикладах. Залишились дві розбіжності, обидві описані в README:

- браузер «відновлює» `<b>` у наступному `<li>`, а ми ні;
- `∉` у нас лишається як написано, а браузер з нашою таблицею зробив би з нього `¬in;`. Це я вирішив сам: напіврозкодована сутність гірша за незмінену.

Також оновив README, `06-entities.html` і `stress/README.md`.

Лишилося два кроки: хостинг і посилання на цю розмову в `ai_help/claude_chat.txt` (там поки заглушка). Для хостингу раджу GitHub Pages: проєкт — це статичні файли без збірки, тож він піднімається прямо з репозиторію, який ти все одно здаватимеш.

окей, тепер обираємо хостинг, під такий проєкт, щоб бажано було обрати?

Раджу **GitHub Pages**. Проєкт — це статичні файли (HTML, CSS, JS) без збірки та без сервера, тож підійде будь-який статичний хостинг. GitHub Pages тут найлогічніший:

- **Код і демо в одному місці.** Репозиторій ти все одно здаватимеш. Рецензент відкриває його, а посилання на робоче демо лежить поруч, у README.
- **Нічого налаштовувати.** Без акаунтів на сторонніх сервісах і без конфігів: пушнув — і сайт оновився.
- **Безкоштовно і без обмежень, які тут відчуються.** Сайти на ньому не «засинають», як на деяких безкоштовних тарифах. Адреса виду `username.github.io/html2json` виглядає охайно.

Альтернативи:

- **Netlify або Cloudflare Pages.** Теж безкоштовні та швидкі. Перевага — попередній перегляд кожного pull request окремим посиланням. Для тестового завдання це зайве, але якщо вже маєш там акаунт, це нормальний вибір.
- **Vercel.** Зручний для React і Next.js, для статики працює так само. Знаю, що на Fiverr ти

робиш лендинги на React, тож якщо вже ним користуєшся, різниці майже немає.

Чого не брати: Render, Railway та інші сервіси для бекенду. Вони для серверних застосунків, а безкоштовні сервери там засинають і довго прокидаються.

Як підняти на GitHub Pages:

1. Створи публічний репозиторій, наприклад `html2json`, і залий туди вміст архіву (саму папку, не zip).
2. Відкрий Settings → Pages → Source: «Deploy from a branch» → гілка `main`, папка `/(root)` → Save.
3. Через хвилину-дві сайт з'явиться за адресою `https://<username>.github.io/html2json/`.

Один момент: за кореневою адресою відкриється `index.html`, тобто оригінальна сторінка завдання, бо її ми не змінюємо. Наше демо буде за адресою `.../html2json/demo.html`. У README я б дав два посилання, «Demo» і «Original task page», щоб рецензент одразу

потрапив куди треба.

Коли буде адреса, скинь її, і я впишу посилання в README.

з приватного репу він цей сервіс працювати може?

Виконано пошук в інтернеті

Залежить від тарифу GitHub:

- **Безкоштовний GitHub Free** — ні. GitHub Pages тут працює лише з публічних репозиторіїв.
- **GitHub Pro (платний)** — так, Pages працює і з приватних.

Тобі, як студенту, Pro може дістатися безкоштовно через **GitHub Student Developer Pack**. Подаєш заявку з університетською поштою або студентським квитком, підтвердження зазвичай займає від кількох днів до тижня.

Навіть із приватного репозиторію **сам сайт буде публічним**: будь-хто за посиланням побачить демо і зможе відкрити `html2json.js` через інструменти браузера. Приватним лишиться тільки репозиторій, тобто історія комітів, тести та README.

Якщо репозиторій має бути приватним і без оплати, бери **Netlify**, **Vercel** або **Cloudflare Pages**. Вони безкоштовно підключаються до приватних репозиторіїв GitHub і викладають сайт після кожного пушу. Для статичного проєкту налаштування займає хвилину: build command лишаєш порожнім, publish directory — корінь (`/`).

Якщо ж репозиторій ти все одно віддаватимеш рецензентам, простіше зробити його публічним і лишитися на GitHub Pages. Перед цим перевір, чи завдання не просить тримати рішення приватним. Тоді вже Netlify або Vercel із приватного репозиторію, а рецензентам дати доступ через Settings → Collaborators.

Sources:

- About GitHub's plans
- Creating a GitHub Pages site
- GitHub Pages with Private Repo — community discussion